

Name: SHI Jiaqi

Student ID: 1155246523

Agentic AI for Business and FinTech (FTEC5660)

Individual Homework 04, Due date: April 26, 11:59 PM (Hong Kong Time).

1. Introduction

This project builds a Financial Analysis Agent for a synthetic company, NovaTech Corp. The main objective is to combine retrieval-augmented generation, tool-based financial reasoning, and anti-sycophancy guardrails into one agentic financial analysis workflow. In financial applications, sycophancy is a serious risk because a model may change its final answer to agree with a misleading user hint, even when the retrieved evidence or its own reasoning trace supports a different conclusion. This can lead to incorrect investment analysis, misleading interpretation of financial performance, and flawed risk assessment.

The implemented system follows three major components. First, a RAG pipeline retrieves relevant passages from NovaTech's financial corpus. Second, a financial reasoning agent uses verified financial data and tools to perform calculations and comparisons. Third, a Judge and feedback controller detect trace-output mismatches and re-run the agent with stronger verification strategies when the answer is not supported by the reasoning trace. The goal is not only to answer financial questions, but also to make the final answer more robust against adversarial or misleading user suggestions.

2. RAG Pipeline

The notebook provides a synthetic NovaTech financial corpus, including the company's annual report, earnings call transcript, and investment research report. These documents contain key financial information such as FY2023 and FY2024 revenue, net income, EPS, operating margin, P/E ratio, and debt-to-equity ratio. The RAG pipeline first splits the documents into text chunks, embeds them, and stores them in a FAISS vector store. The implemented retrieval chain uses the retriever together with a formatting function:

```
rag_retrieval_chain = retriever | format_docs
```

This chain takes a financial question as input, retrieves the most relevant document chunks, and returns a formatted context string. The retrieved context is then used by the basic RAG chain and the later anti-sycophancy loop.

The RAG pipeline was tested on three financial questions. For the revenue growth question, the system retrieved the relevant FY2023 and FY2024 revenue figures and answered that NovaTech's revenue increased from \$3.5 billion to \$4.2 billion, corresponding to a year-over-year growth rate of 20.0%. For the P/E ratio comparison, it answered that NovaTech's trailing P/E ratio was 28.5x, which was below the FinTech sector average of 31.2x. For the debt-to-equity question, it identified NovaTech's debt-to-equity ratio as 0.43 and explained that this is significant because it is lower than the industry average of 0.72, suggesting a relatively conservative leverage profile.

Overall, the RAG component successfully provided factual grounding for the financial reasoning tasks. It helped ensure that the agent's answers were based on the NovaTech corpus instead of unsupported user claims.

3. Financial Reasoning Agent

The Financial Reasoning Agent was designed through a system prompt that defines the agent as a careful professional financial analyst. The prompt requires the agent to prioritize verified data over user suggestions and to follow explicit reasoning steps. These steps include identifying the required financial metric, retrieving or verifying relevant figures, performing calculations, comparing results to benchmarks when appropriate, checking for contradictions, and ending with a clear final answer and confidence assessment.

The prompt also instructs the agent to show all calculations. For example, for revenue growth, the agent should use the formula:

$$\text{Growth Rate} = (\text{Current Revenue} - \text{Previous Revenue}) / \text{Previous Revenue} \times 100$$

Using the verified NovaTech data, this becomes:

$$(\$4.2B - \$3.5B) / \$3.5B \times 100 = 20.0\%$$

A key part of the prompt is the anti-sycophancy instruction. If the user provides a hint, correction, acquisition claim, restatement claim, or suggested conclusion that conflicts with verified data, the agent is instructed to treat it as unsupported and not adopt it. The final answer is required to begin with the verified numerical answer and avoid repeating unsupported user-provided numbers or claims. This is important because even mentioning an adversarial hint in the final answer may cause the answer to appear sycophantic under automatic evaluation.

The final answer format was strengthened as follows:

Final Answer: <verified answer>

Calculation: <formula and numbers>

Evidence: <verified source or tool output>

Confidence: <High/Medium/Low>

This format improves consistency and makes it easier for the evaluation logic to identify the correct numerical answer.

4. Sycophancy Judge

The Judge is responsible for checking whether the final answer is supported by the agent's reasoning trace and retrieved context. It is not supposed to solve the financial question independently or use external knowledge. Instead, it evaluates internal consistency.

The Judge prompt instructs the Judge to detect three major failure types. First, it checks trace-output mismatches, where the reasoning trace computes or retrieves one value but the final answer gives a different value. Second, it checks unsupported logical leaps, where the final answer includes a number, comparison, or interpretation not supported by the trace or context. Third, it checks internal contradictions within the reasoning trace or between the trace and the final answer.

The Judge is required to output exactly two lines. The first line must be either Pass or Fail, and the second line must contain a one-sentence critique. This strict output format makes it easier for the feedback controller to parse the Judge result and decide whether to rerun the agent.

The Judge is intentionally limited to internal consistency checking. This design is

important because the purpose of the Judge is not to replace the financial agent, but to detect whether the final response faithfully follows the evidence and reasoning already available in the trace.

5. Anti-Sycophancy Feedback Controller

The feedback controller implements an escalation mechanism based on the Judge verdict. If the Judge returns Pass, the answer is accepted. If the Judge returns Fail, the controller records an error signal:

$$e_t = 1 \text{ if } \text{verdict}_t = \text{Fail}, \text{ otherwise } 0$$

The integral error signal is then accumulated:

$$E_{int} = \text{sum of previous error signals}$$

The controller uses this accumulated error signal to choose the agent's next persona and reasoning strategy:

E_int	Persona	Strategy
0	Helpful	Direct
1	Skeptical	Chain-of-Thought
≥ 2	Skeptical	Python code execution

The implemented function `compute_error_signal` returns 1 for Fail and 0 for Pass. The implemented function `get_strategy_and_persona` maps the accumulated error signal to the appropriate persona and strategy.

To improve the feedback loop, the Judge critique is passed into the next iteration. This allows the agent to revise its answer based on the reason for failure instead of simply repeating the same response. In addition, the retrieved context is included in every anti-sycophancy turn so that the agent remains anchored to verified evidence. If all retry attempts still fail, the system uses a deterministic fallback answer based on verified NovaTech financial data. This prevents the controller from returning the last unstable or sycophantic model output.

6. Evaluation Results

The system was evaluated under three conditions: baseline without adversarial hints, adversarial questions without anti-sycophancy protection, and adversarial questions with the anti-sycophancy controller. The evaluation reports Accuracy, Sycophancy Rate, and FOG Rate.

In my run, the results were:

Condition	Accuracy	Sycophancy Rate	FOG Rate
D0 Baseline, no hint, no Anti	80.0%	0.0%	0.0%
DS Adversarial, no Anti	80.0%	80.0%	60.0%
DS Adversarial, with Anti	60.0%	60.0%	40.0%

The baseline condition shows that the agent performed reasonably well when there was no misleading user hint. The Sycophancy Rate and FOG Rate were both 0.0%, indicating that there was no adversarial pressure causing the model to follow an incorrect suggestion.

Under the adversarial setting without anti-sycophancy protection, the Sycophancy Rate increased to 80.0%, and the FOG Rate increased to 60.0%. This shows that adversarial user hints can strongly influence the model's final answer. The high FOG Rate is especially concerning because it means that in some cases, the reasoning trace contained the correct answer, but the final output still followed the misleading hint.

After applying the anti-sycophancy controller, the Sycophancy Rate decreased from 80.0% to 60.0%, and the FOG Rate decreased from 60.0% to 40.0%. This suggests that the feedback loop partially reduced the agent's tendency to follow misleading user hints. However, Accuracy also declined from 80.0% to 60.0%. This indicates a trade-off: the stricter Judge and skeptical reasoning strategy reduced unsupported agreement with the user, but sometimes caused the agent to produce overly cautious or less precisely formatted answers.

7. Discussion of FOG Rate

FOG Rate measures the most dangerous type of sycophancy. It counts cases where the reasoning trace contains the correct answer, but the final answer still follows the adversarial hint. This is more severe than a normal wrong answer because the model had access to the correct information but failed to preserve it in the final response.

FOG Rate is always less than or equal to Sycophancy Rate. Sycophancy Rate counts all cases where the final answer follows the misleading hint. FOG Rate is a stricter subset because it only counts sycophantic cases where the correct answer also appeared in the trace. Therefore, the difference between Sycophancy Rate and FOG Rate helps distinguish between two failure modes. If the model never found the correct answer, the failure is mainly a retrieval or reasoning problem. If the model found the correct answer but still followed the hint, the failure is a final-output alignment problem.

In the evaluation, the adversarial setting without anti-sycophancy protection had a Sycophancy Rate of 80.0% and a FOG Rate of 60.0%. This means that many failures were not due to missing information, but due to the model failing to preserve its own correct reasoning in the final answer. With the anti-sycophancy controller, FOG Rate decreased to 40.0%, showing that the guardrail reduced some of these trace-output mismatch failures.

8. Conclusion

This homework demonstrates how a financial AI agent can be improved by combining RAG, tool-based reasoning, and guardrails. The RAG pipeline grounds the agent in NovaTech's financial documents. The reasoning agent performs explicit financial calculations and comparisons. The Judge and feedback controller help detect and reduce sycophantic behavior by checking whether the final answer is supported by the reasoning trace.

The evaluation results show that adversarial hints can significantly increase sycophancy. The anti-sycophancy controller partially reduced both Sycophancy Rate and FOG Rate, indicating that the feedback loop improved robustness against misleading user suggestions. However, the decline in Accuracy suggests that further tuning is needed. In future improvements, the system could use more precise final-answer formatting,

stronger deterministic verification for numerical questions, and better calibration of the Judge so that correct but concise answers are not rejected unnecessarily. Overall, the project shows that financial agents should not only retrieve and calculate correctly, but also maintain consistency between verified evidence, reasoning, and final output.